

CIS 5357

Computing for Data Analytics

Week 8

7 MARCH 2023

Part 2: Data Analysis

Week	Lecture	Format	Topic	Assignments	Due Date
PART 2		DATA ANALYSIS			
8	3/7	in person	Introduction to Jupyter notebooks & pandas		
9	3/14	no lecture	SPRING BREAK – no class		
10	3/21	virtual	Pandas for data exploration	Coding assignment (10%)	3/28
11	3/28	in person	Pandas for data manipulation		
12	4/4	virtual	Dataset joining & filter	Coding assignment (10%)	4/11
13	4/11	in person	Data aggregation		
14	4/18	virtual	Dig case part 1	Writing assignment (10%)	4/25
15	4/25	no lecture	Dig case part 2; course wrap up – no synchronous lecture		
16	5/2	no lecture	Final exam project	Final Exam (20%)	5/2

This week

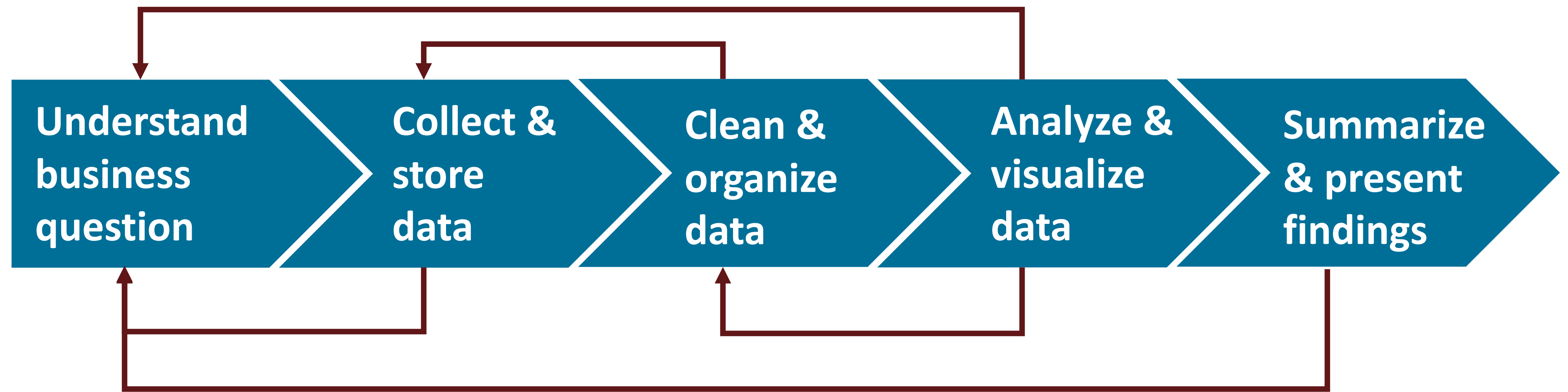
- Data analysis process
- Introduce the Dig case study
- Jupyter notebooks
- Pandas library: series and DataFrames
- Reading and writing files with pandas

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions

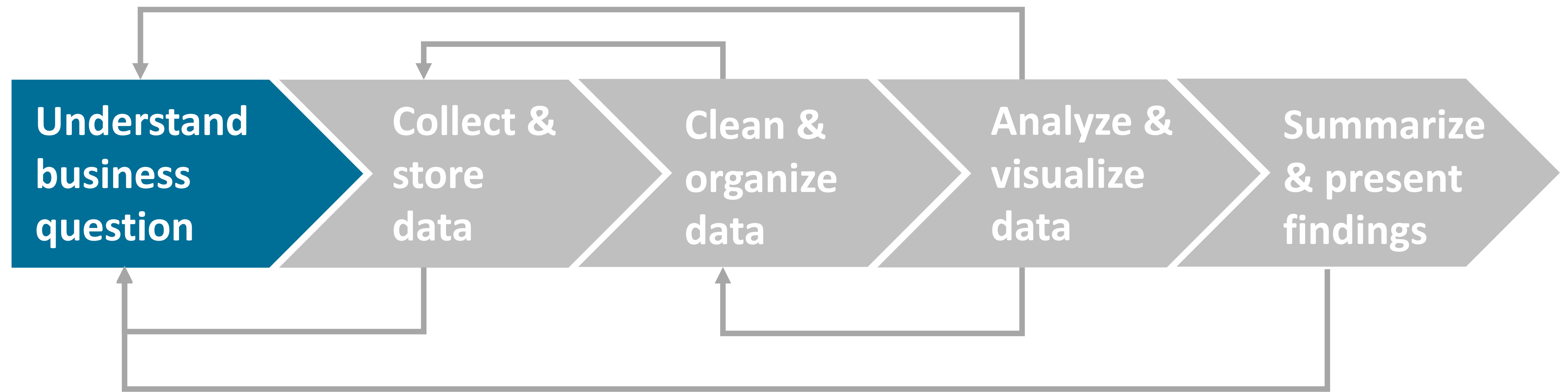
Data Analytics

the **process** of analyzing **raw data** in order to draw out **meaningful, actionable insights**, which are then used to inform and drive **business decisions**



Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions



Define objectives

Work with your customer and other stakeholders to understand and identify the business problem.

Formulate questions that define the business goals that the data analysis techniques can target.

- How much or how many? (aggregation, regression)
- Which category? (ranking, classification)
- Which group? (grouping, clustering)
- Is this unusual? (anomaly detection)
- Which option should be taken? (forecasting, ROI)

Identify data sources

Find the **relevant data** that helps you answer the questions that define the objectives of the project.

- Data that's relevant to the question
- Data that's an accurate measure of your target

In some cases, an existing system might not have the data it needs to address a problem and achieve a project goal. In this situation, you might need to **find external data sources** or **update your systems to collect new data**.

Dig case study

- **Dig is a restaurant chain** in New York City with a winning concept: deliver a delicious vegetable-first menu **sourced directly** from farms, price each meal **affordably**, and have a team of **skilled chefs** prepare everything on site.
- Founded in 2011, it has grown from a single restaurant to **multicity reach**. This evolution involved significant changes in Dig's menu and its operations—for example, making its goods available on **delivery apps**, launching a **catering service**, and developing a **delivery-specific menu**.
- As it grew, the company **collected data on aspects of its operations and management**, but the focus was on perfecting the product and defining Dig's identity. Analyses using data were often **one-off and involved painstakingly piecing together disparate datasets** to answer specific questions, rather than relying on more robust systems and reporting tools, such as dashboards.

New strategy: data-driven decisions

GOAL: use data to inform decisions at every stage in the value chain

- With new restaurants opening, it is less sustainable to rely on intuition to make decisions

Available data:

- Restaurants.csv
- Items.csv
- Simplified orders.csv
- Summarized orders.csv

Data in textbook files you downloaded in Week 1
[python-for-mbas/Part 2/ raw data](#)

Restaurants.csv

data pertaining to each restaurant (9 rows, 7 columns)

RESTAURANT_ID	NAME	ADDRESS	LAT	LONG	OPENING_DATE	DELIVERY_START
R10001	Columbia	2884 Broadway, New York, NY 10025	40.811470	-73.961230	8/9/2014	2017-01-01
R10002	Midtown	1379 6th Ave, New York, NY 10019	40.763640	-73.977960	3/19/2013	2018-05-01
R10005	Flatiron	40 W 25th St, New York, NY 10010	40.743600	-73.991070	11/14/2013	2016-03-05
R10008	Williamsburg	45 S 3rd St, Brooklyn, NY 11249	40.713749	-73.965782	10/12/2015	2017-01-01
R10004	NYU	109 Macdougall St, New York, NY 10012	40.729930	-74.000820	1/10/2014	2017-01-01

Items.csv

lists the items sold (32 rows, 3 columns)

ITEM_ID	ITEM_NAME	ITEM_TYPE
17	Farro with Summer Vegetables	Bases
139	Spindrift Lemon	Drinks
15	Classic Brown Rice	Bases
136	Kombucha	Drinks
18	Cauliflower with Garlic and Parmesan	Market Sides

Simplified orders.csv

one line per order in the one-year period spanned by the dataset
(2.4M rows, 10 columns)

ORDER_ID	DATETIME	RESTAURANT_ID	TYPE	DRINKS	COOKIES	MAIN	BASE	SIDE_1	SIDE_2
O1820060	2018-10-11 17:25:50	R10002	IN_STORE	1.0	2.0	NaN	NaN	NaN	NaN
O1011112	2018-05-31 11:35:00	R10003	IN_STORE	0.0	0.0	NaN	NaN	NaN	NaN
O752854	2018-04-21 18:12:57	R10001	DELIVERY	0.0	2.0	I0	I7	I15	I14
O2076864	2018-11-17 12:50:52	R10005	PICKUP	1.0	0.0	I0	I5	I9	I12
O1988898	2018-11-04 18:37:24	R10008	IN_STORE	0.0	0.0	I1	I7	I9	I9

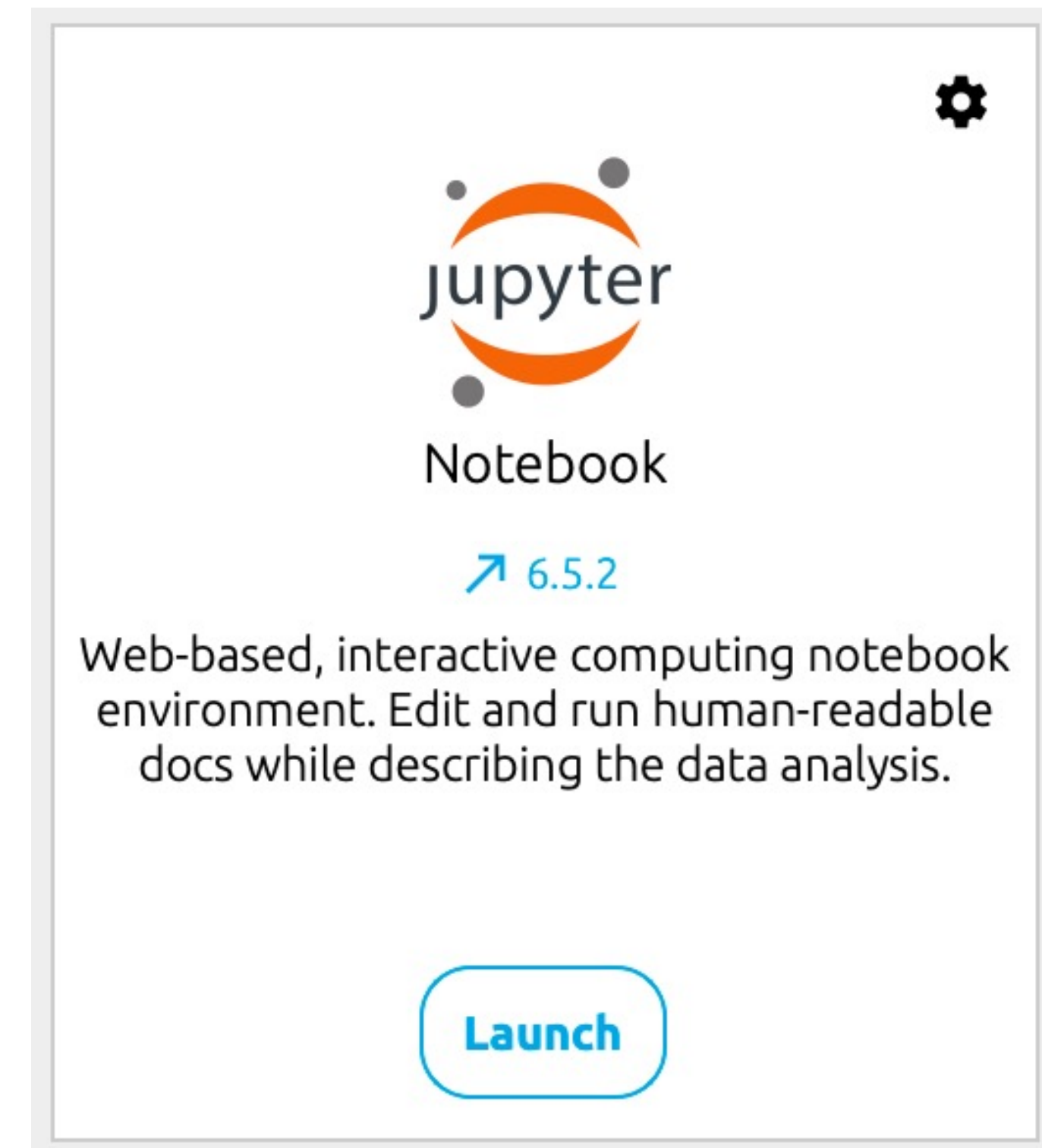
Summarized orders.csv

aggregates the orders file to a daily level—it contains one row per restaurant per day (2,807 rows, 4 columns)

location	day	number of orders	percentage of deliveries
Bryant Park	2018-01-01	373	0.0
Bryant Park	2018-01-02	789	0.0
Bryant Park	2018-01-03	818	0.0
Bryant Park	2018-01-04	782	0.0
Bryant Park	2018-01-05	719	0.0

Jupyter notebook

- A notebook integrates code and its output into a single document
- You can also add explanations, formulas, charts, to make your work more transparent, understandable, repeatable, and shareable

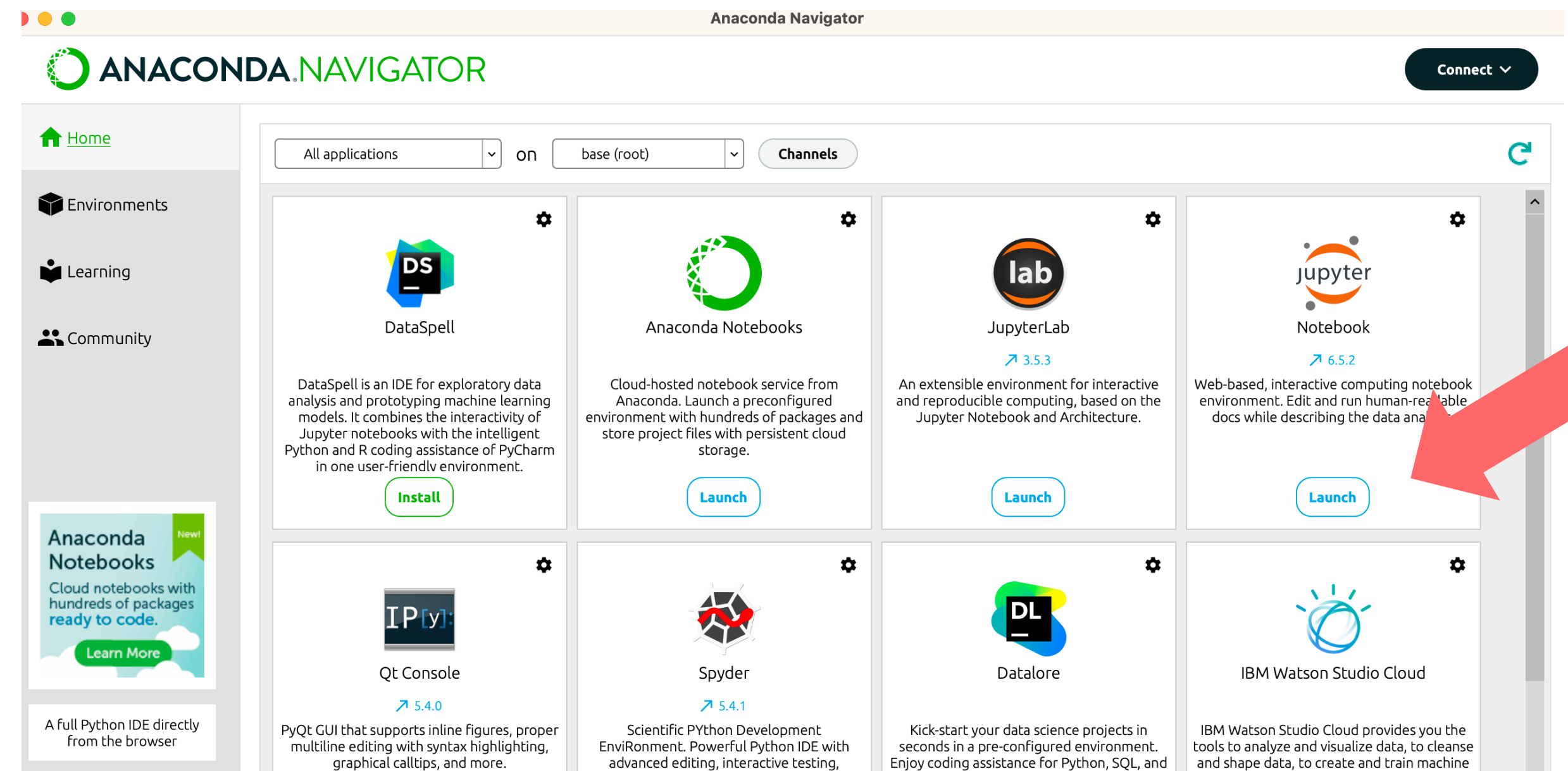


Launch Jupyter notebook

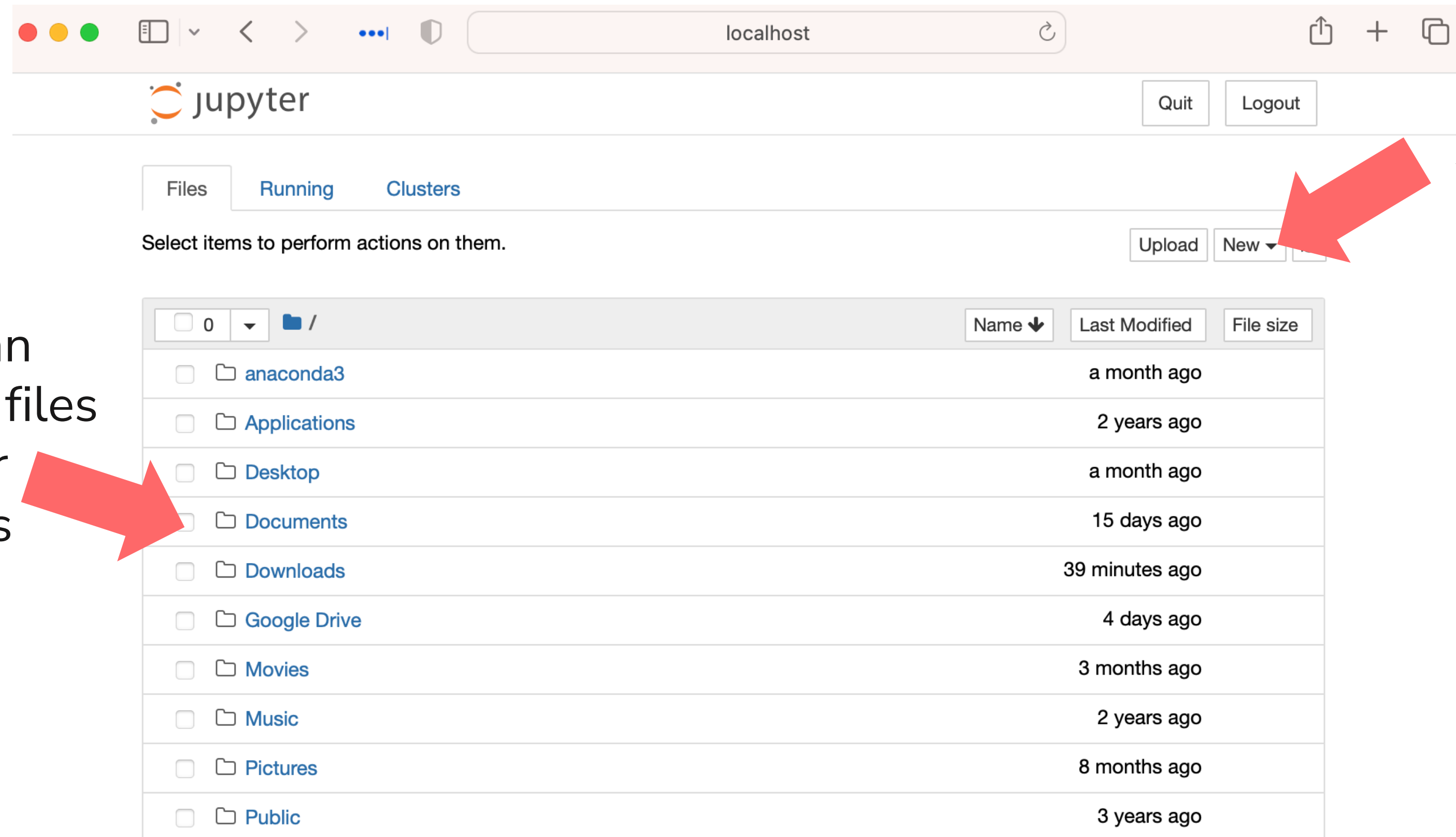
Launch from Anaconda navigator

OR

Open a command line prompt and type **jupyter notebook** in the terminal

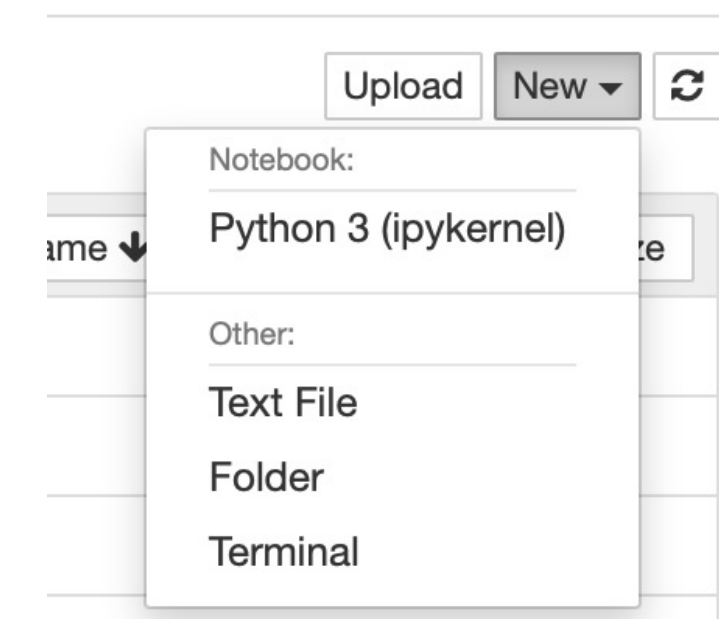


Jupyter browser window

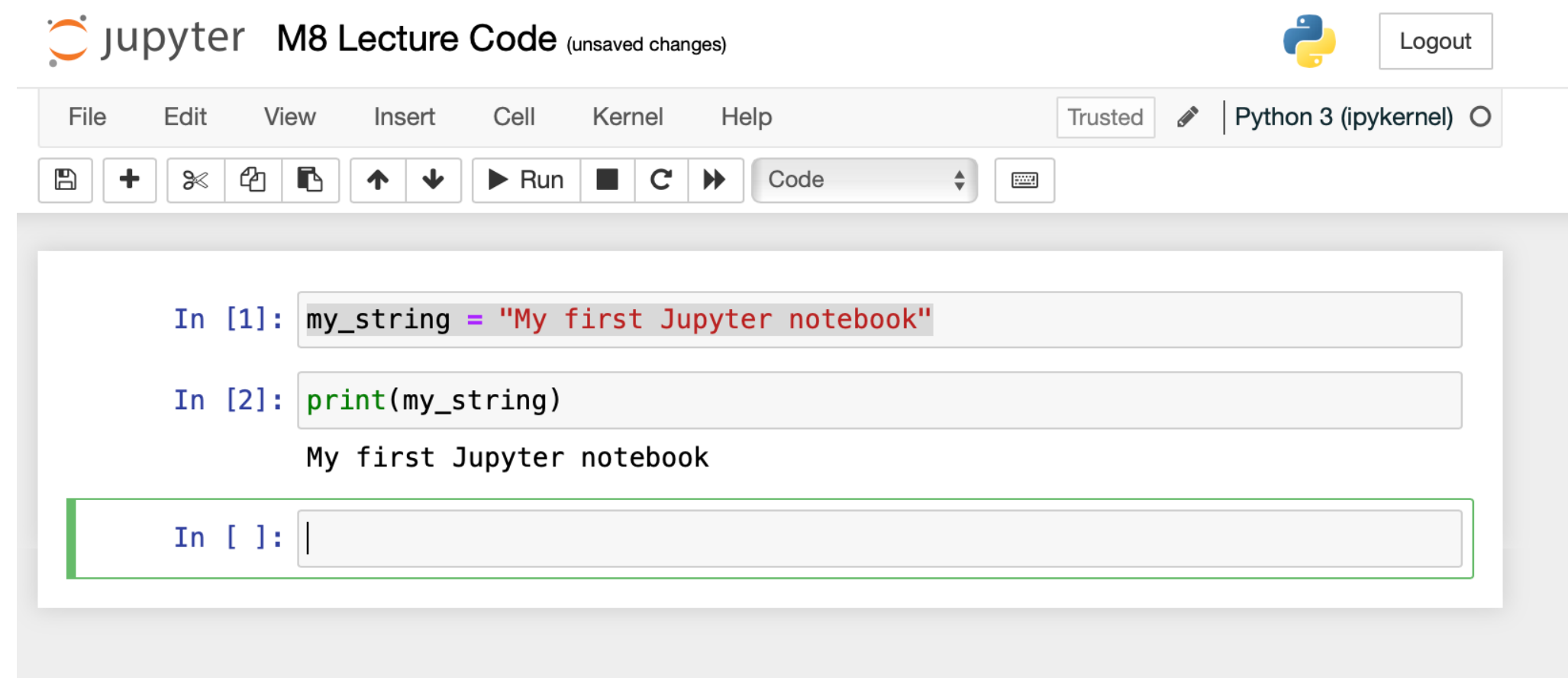


You can locate files in your folders

And also create new notebook files using the dropdown



Jupyter notebook cells

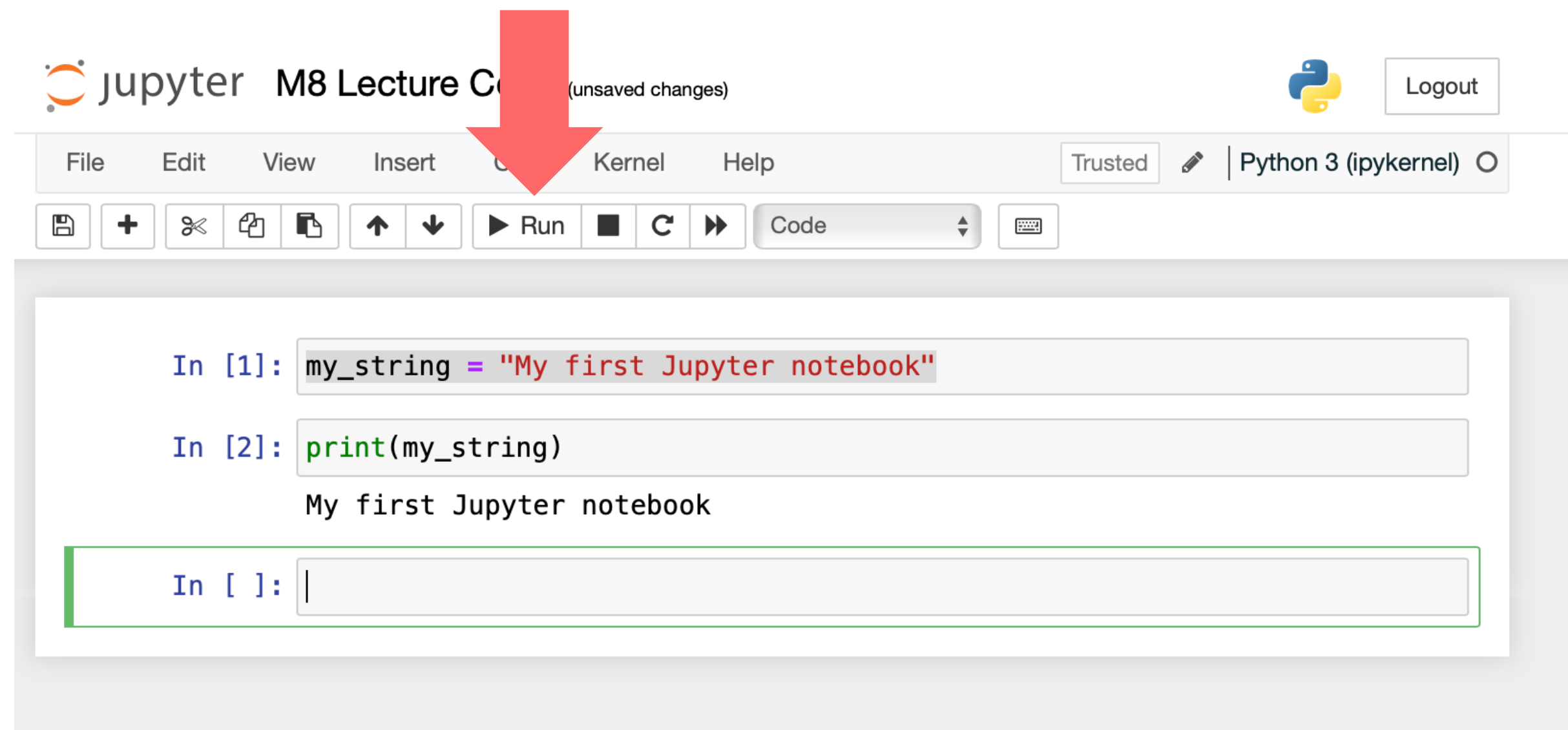


* If you close the command line window while *Jupyter* is running in your browser your kernel connection will be lost & you won't be able to run any code in the notebook anymore.

- A Jupyter Notebook file is made up of one or more **cells** that contain code
- All the cells in a notebook run in the **same Python workspace** so a variable defined in one cell, will exist in another cell
- If you created a whole separate notebook, however, it would run in its own separate kernel, so you could run different analyses side by side

Running code

Click the run button

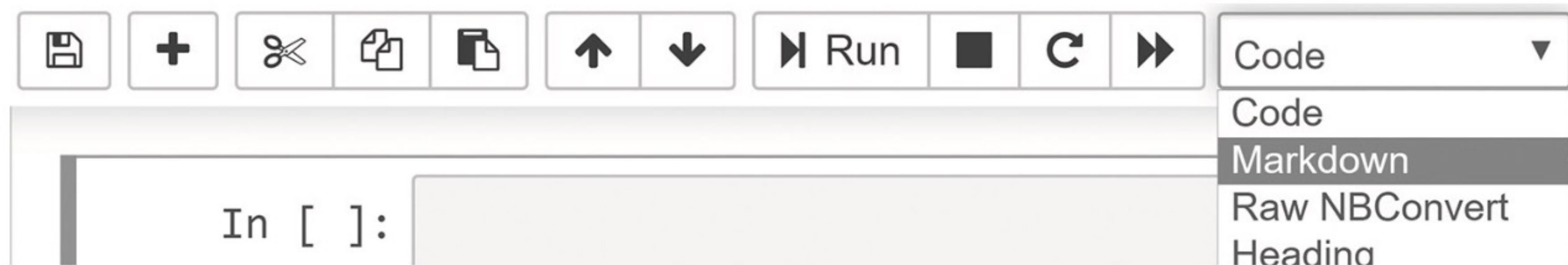


Shortcuts :

- ***Ctrl + Enter*** to run a selected cell—this runs a cell, and keeps that cell selected
- ***Shift + Enter*** will run the cell and select the cell below
- ***Alt + Enter*** will run the cell and create an empty cell below the cell that just ran

Markdown cells

Markdown cells do not contain code—instead, they contain annotations—much like comments but formatted.



Let's chat

When would you use a
notebook versus terminal?

Common libraries for data analysis

Pandas

NumPy

SciPy

SciKit-Learn

Matplotlib

Seaborn

Plotly

Pandas library

```
import pandas as pd
```

Provide two new data types to work with data:

1. **Series**: can store columns
2. **DataFrame**: collection of multiple series (columns) together with their column titles to give us a full table

DataFrame structure

The diagram illustrates the structure of a DataFrame. It features a table with 9 rows and 8 columns. The first column contains row indices from 0 to 8. The subsequent columns are labeled: FIRST_NAME, LAST_NAME, YEAR, HOME_STATE, AGE, CALC_101_FINAL, and ENGLISH_101_FINAL. An arrow labeled 'Column names' points to the header row. An arrow labeled 'Missing values' points to a 'NaN' value in the ENGLISH_101_FINAL column for row 1. An arrow labeled 'Row index' points to the first column. Multiple arrows labeled 'Individual series (columns)' point to each of the seven data columns. The table data is as follows:

	FIRST_NAME	LAST_NAME	YEAR	HOME_STATE	AGE	CALC_101_FINAL	ENGLISH_101_FINAL
0	Daniel	Smith	1	NY	18	90.0	80.0
1	Ben	Leibstrom	1	NY	19	80.0	NaN
2	Kavita	Kanabar	1	PA	19	NaN	NaN
3	Linda	Thiel	4	CA	22	60.0	40.0
4	Omar	Reichel	2	OK	21	70.0	50.0
5	Jane	OConner	2	HI	19	NaN	NaN
6	Felicia	Rao	3	NY	20	NaN	NaN
7	Rachel	Crock	1	FL	17	NaN	60.0
8	Bob	McDonald	1	FL	18	98.0	65.0

Download code to follow along



Read

- Textbook chapter 5
- [5 Analytical Skills that Every Data Analyst Should Have](#) ↓
- Explore challenges and drawbacks of Excel
 - [How the London whale debacle is partly the result of an error using Excel](#) ↗
 - [Why using Microsoft's tool caused Covid-19 results to be lost](#) ↗
- [Practice code for lecture](#) ↓
- Lecture slides (coming soon!)

- Technical documentation and other references/resources
 - [Python Scripts vs. Jupyter Notebooks: Pros and Cons](#) ↗
 - [Jupyter notebook documentation](#) ↗
 - [Working with in Jupyter Notebooks in Visual Studio Code](#) ↗
 - [Pandas data structures: series & dataframes](#) ↗
 - [Python documentation: pickle module](#) ↗
 - [Pandas read_excel](#) ↗ and [read_csv](#) ↗

Creating a DataFrame

```
customers = (  
    {'FIRST_NAME': ['Khue', 'Wasim', 'Jose', 'Marie',  
                  'Jewel', 'Brianna', 'Costa', 'Rachel'],  
     'LAST_NAME': ['Nguyen', 'Ansari', 'Sanchez', 'Reyes',  
                  'Neal', 'Brown', 'George', 'Crock'],  
     'STATE': ['NY', 'TX', 'PA', 'CA', 'OK', 'HI', 'NY', 'TX'],  
     'AGE': [18, 39, 19, 27, 21, 49, 20, 38] }  
)  
df_customers = pd.DataFrame(customers)
```

✓ 0.0s

It's possible to create a DataFrame directly in pandas using a dictionary of lists, in which each entry in the dictionary corresponds to a column and contains a list of entries.

df_customers

✓ 0.0s

	FIRST_NAME	LAST_NAME	STATE	AGE
0	Khue	Nguyen	NY	18
1	Wasim	Ansari	TX	39
2	Jose	Sanchez	PA	19
3	Marie	Reyes	CA	27
4	Jewel	Neal	OK	21
5	Brianna	Brown	HI	49
6	Costa	George	NY	20
7	Rachel	Crock	TX	38

Accessing a column—2 options

1. put the name of the column in square brackets:

```
df_customers['FIRST_NAME']
```

✓ 0.0s

```
0    Khue
1    Wasim
2     Jose
3    Marie
4    Jewel
5  Brianna
6     Costa
7    Rachel
Name: FIRST_NAME, dtype: object
```

* Note this returns a series

Accessing a column—2 options

2. put a dot after the name of the DataFrame, followed by the name of the column.

```
df_customers.FIRST_NAME
```

✓ 0.0s

```
0    Khue
1    Wasim
2     Jose
3    Marie
4    Jewel
5  Brianna
6    Costa
7    Rachel
Name: FIRST_NAME, dtype: object
```

* Note column names are case sensitive

Selecting multiple columns

Use embedded square brackets

```
df_customers[['FIRST_NAME', 'LAST_NAME']]
```

✓ 0.0s

	FIRST_NAME	LAST_NAME
0	Khue	Nguyen
1	Wasim	Ansari
2	Jose	Sanchez
3	Marie	Reyes
4	Jewel	Neal
5	Brianna	Brown
6	Costa	George
7	Rachel	Crock

* Note this returns a dataframe

Concatenate multiple columns

Using dot notation

```
selected_columns = df_customers.FIRST_NAME.str.cat(df_customers.LAST_NAME, sep=' ')\nprint(selected_columns)
```

✓ 0.0s

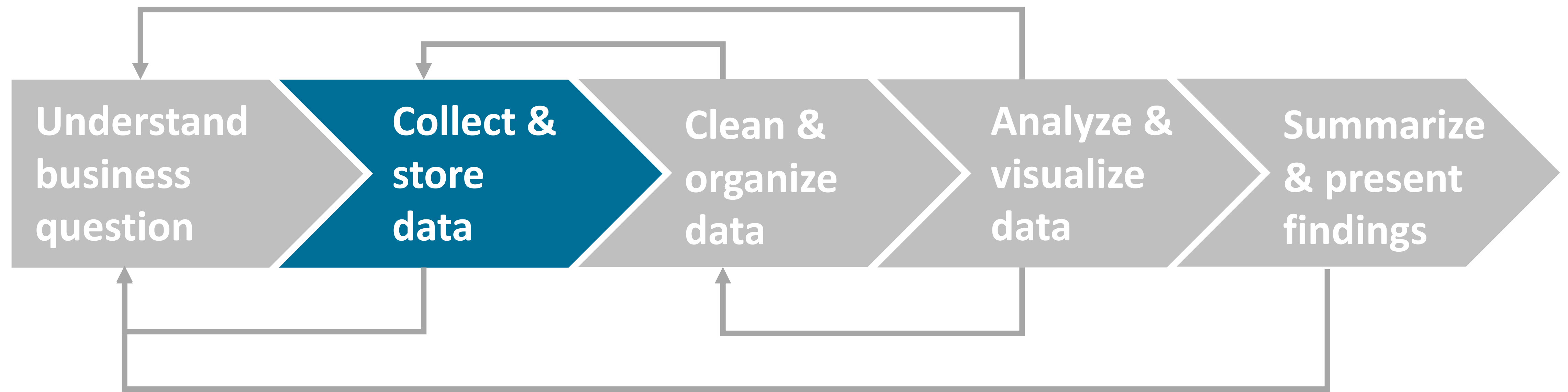
```
0    Khue Nguyen\n1    Wasim Ansari\n2    Jose Sanchez\n3    Marie Reyes\n4    Jewel Neal\n5    Brianna Brown\n6    Costa George\n7    Rachel Crock
```

Name: FIRST_NAME, dtype: object

* Still returns a series

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions



Reading data from a file

It's also possible to create a DataFrame directly from a file

```
pd.read_excel('path/filename.xlsx', sheet_name='sheet_name') #by default reads 1st sheet  
pd.read_csv(path/filename.csv)
```

Reading data from a file

It's also possible to create a DataFrame directly from a file

```
pd.read_excel('path/filename.xlsx', sheet_name='sheet_name') #by default reads 1st sheet  
pd.read_csv(path/filename.csv)
```

Optional arguments (see more in documentation):

- *usecols*: returns a subset of the columns
- *nrows*: specify a number of rows to read
- *skiprows*: specify rows to skip over
- *header = none*: to consider the first row as a data record

View the first 5 rows with `.head()`

```
df_summarized_orders = pd.read_csv(  
    'raw data/Summarized orders.csv')  
df_summarized_orders.head()
```

✓ 0.0s

	location	day	number of orders	percentage of deliveries
0	Bryant Park	2018-01-01	373	0.0
1	Bryant Park	2018-01-02	789	0.0
2	Bryant Park	2018-01-03	818	0.0
3	Bryant Park	2018-01-04	782	0.0
4	Bryant Park	2018-01-05	719	0.0

- `DataFrame.head()` function shows you the first 5 rows of data in the DataFrame. The opposite is `DataFrame.tail()`, which gives you the last 5 rows
- Pass in a number and Pandas will print out the specified number of rows

Rename columns

If column names include spaces or are rather long we can rename them to make it easier to read and type.

```
df_summarized_orders.columns = ['RESTAURANT_NAME',  
                                'DATE', 'NUM_ORDERS',  
                                'PERC_DELIVERY']  
df_summarized_orders.head()
```

✓ 0.0s

	RESTAURANT_NAME	DATE	NUM_ORDERS	PERC_DELIVERY
0	Bryant Park	2018-01-01	373	0.0
1	Bryant Park	2018-01-02	789	0.0
2	Bryant Park	2018-01-03	818	0.0
3	Bryant Park	2018-01-04	782	0.0
4	Bryant Park	2018-01-05	719	0.0

Data types and file size

```
df_summarized_orders.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2806 entries, 0 to 2805  
Data columns (total 4 columns):  
#      Column          Non-Null Count  Dtype  
---  -  
0     RESTAURANT_NAME    2806 non-null   object  
1     DATE                2806 non-null   object  
2     NUM_ORDERS          2806 non-null   int64  
3     PERC_DELIVERY       2806 non-null   float64  
dtypes: float64(1), int64(1), object(2)  
memory usage: 87.8+ KB
```

```
df_summarized_orders.shape
```

```
(2806, 4)
```



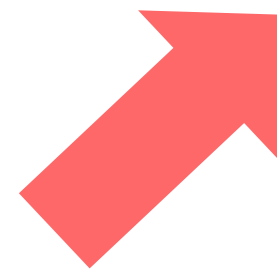
Note there are no
parentheses with *shape*

You can also check the types of each column with `.dtypes` (no parentheses)

Writing out data to .csv

If we want to export our DataFrame back to a file so that we can keep it for future analysis or share it with others; pandas makes this easy with **to_csv()**

```
dataframe_name.to_csv('path/filename.csv', index=False)
```



The string in the function is the file path, followed by the name of the file the DataFrame will be saved



`index=False` tells pandas to save the file without this row index

Writing out data to Excel

If we want to export our DataFrame back to a file so that we can keep it for future analysis or share it with others; pandas makes this easy with **to_excel()**

```
dataframe_name.to_excel('path/filename.xlsx', index=False  
sheetname='Sheet1')
```

The string in the function is the file path, followed by the name of the file the DataFrame will be saved

Options to name the sheet

`index=False` tells pandas to save the file without this row index

Run these & open each of the files

```
df_summarized_orders.to_csv('Summarized orders new.csv', index=False)
```

✓ 0.0s

```
df_summarized_orders.to_excel('Summarized orders new.xlsx', index=False, sheet_name='order')
```

✓ 0.3s

Pickle files

A pickle file allows us to preserve any Python variable, including a DataFrame, and reload it exactly as it was.

```
dataframe_name.to_pickle('path/filename.pickle')
```

Pickle files

A pickle file allows us to preserve any Python variable, including a DataFrame, and reload it exactly as it was.

```
dataframe_name.to_pickle('path/filename.pickle')
```

This file cannot be opened in Excel. It can only be read back into Python, using the `pd.read_pickle()` function

```
pd.read_pickle('path/filename.pickle')
```

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
8 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
7 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
6 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
5 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
4 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
3 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
2 minutes...

Challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

We'll resume in
1 minute...

Let's discuss the challenge

1. Read in the Restaurants.csv file from the textbook
2. Create a new dataframe with just the NAME and ADDRESS
3. Change the column names in the new dataframe to RESTAURANT_NAME and STREET_ADDRESS
4. Export the new dataframe to an Excel file (open the file and confirm the output is as expected)

One possible solution STEP 1

```
# load the restaurants file into a dataframe
df_restaurants = pd.read_csv('raw data/Restaurants.csv')
df_restaurants.head()
```

✓ 0.0s

Py

	RESTAURANT_ID	NAME	ADDRESS	LAT	LONG	OPENING_DATE	DELIVERY_START
0	R10001	Columbia	2884 Broadway, New York, NY 10025	40.811470	-73.961230	8/9/2014	2017-01-01
1	R10002	Midtown	1379 6th Ave, New York, NY 10019	40.763640	-73.977960	3/19/2013	2018-05-01
2	R10005	Flatiron	40 W 25th St, New York, NY 10010	40.743600	-73.991070	11/14/2013	2016-03-05
3	R10008	Williamsburg	45 S 3rd St, Brooklyn, NY 11249	40.713749	-73.965782	10/12/2015	2017-01-01
4	R10004	NYU	109 Macdougall St, New York, NY 10012	40.729930	-74.000820	1/10/2014	2017-01-01

One possible solution STEP 2

```
# create a new dataframe with only name & address  
new_df = df_restaurants[['NAME', 'ADDRESS']]  
new_df.head()
```

✓ 0.0s

	NAME	ADDRESS
0	Columbia	2884 Broadway, New York, NY 10025
1	Midtown	1379 6th Ave, New York, NY 10019
2	Flatiron	40 W 25th St, New York, NY 10010
3	Williamsburg	45 S 3rd St, Brooklyn, NY 11249
4	NYU	109 Macdougall St, New York, NY 10012

One possible solution STEP 3

```
# rename columns  
new_df.columns = ['RESTAURANT_NAME', 'STREET_ADDRESS']  
new_df.head()
```

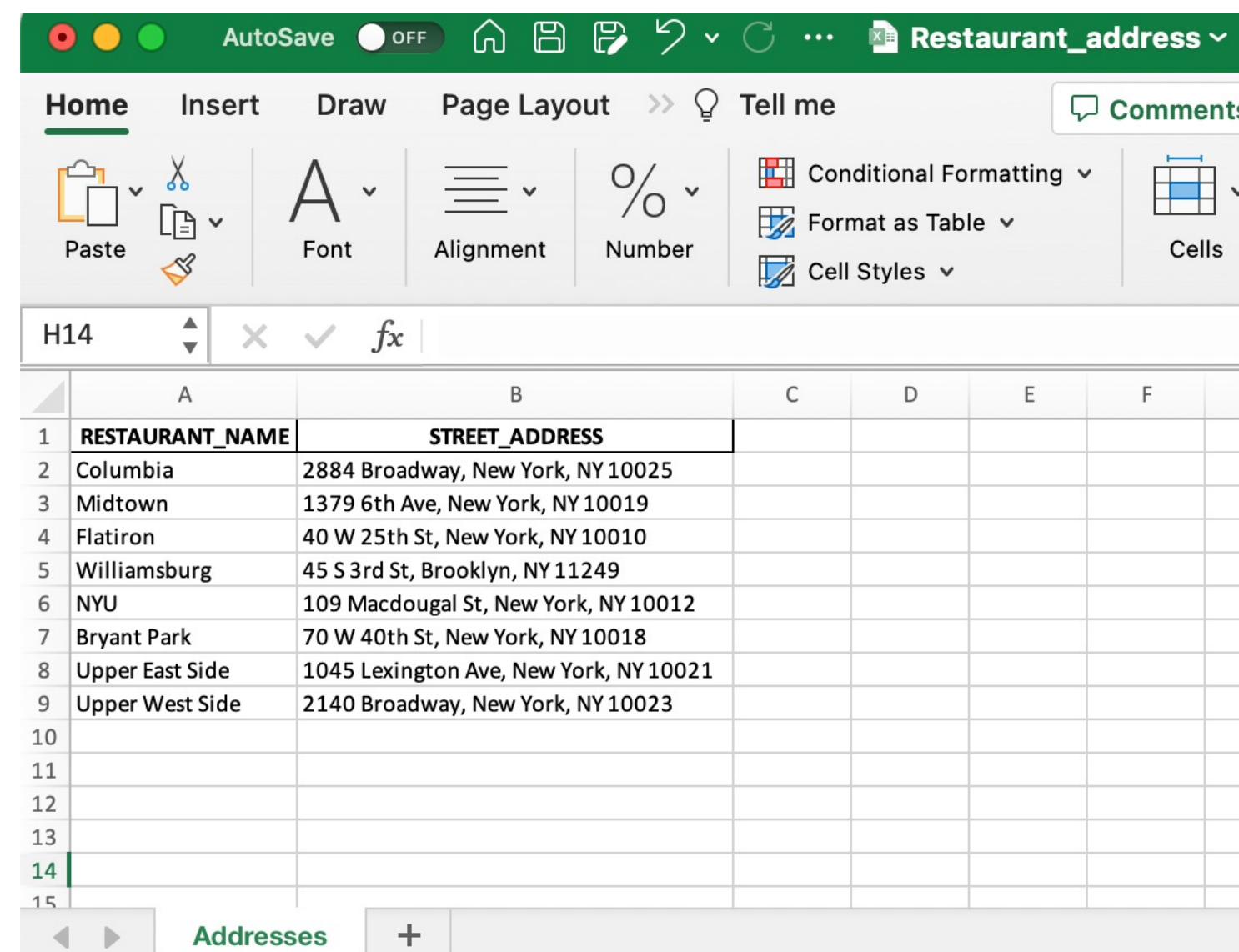
✓ 0.0s

	RESTAURANT_NAME	STREET_ADDRESS
0	Columbia	2884 Broadway, New York, NY 10025
1	Midtown	1379 6th Ave, New York, NY 10019
2	Flatiron	40 W 25th St, New York, NY 10010
3	Williamsburg	45 S 3rd St, Brooklyn, NY 11249
4	NYU	109 Macdougall St, New York, NY 10012

One possible solution STEP 4

```
# export new dataframe to Excel
new_df.to_excel('Restaurant_address.xlsx', index=False, sheet_name='Addresses')
```

✓ 0.2s



	A	B	C	D	E	F
1	RESTAURANT_NAME	STREET_ADDRESS				
2	Columbia	2884 Broadway, New York, NY 10025				
3	Midtown	1379 6th Ave, New York, NY 10019				
4	Flatiron	40 W 25th St, New York, NY 10010				
5	Williamsburg	45 S 3rd St, Brooklyn, NY 11249				
6	NYU	109 Macdougall St, New York, NY 10012				
7	Bryant Park	70 W 40th St, New York, NY 10018				
8	Upper East Side	1045 Lexington Ave, New York, NY 10021				
9	Upper West Side	2140 Broadway, New York, NY 10023				
10						
11						
12						
13						
14						
15						

Open file to confirm

Let's chat

When would you use
Python versus Excel?

Python versus Excel

- Use Python for **large datasets** too big for Excel
- Python is more robust for **complex calculations**
- **Automation** tends to be more efficient in Python
- Python is often better for integration with **operational workflows**

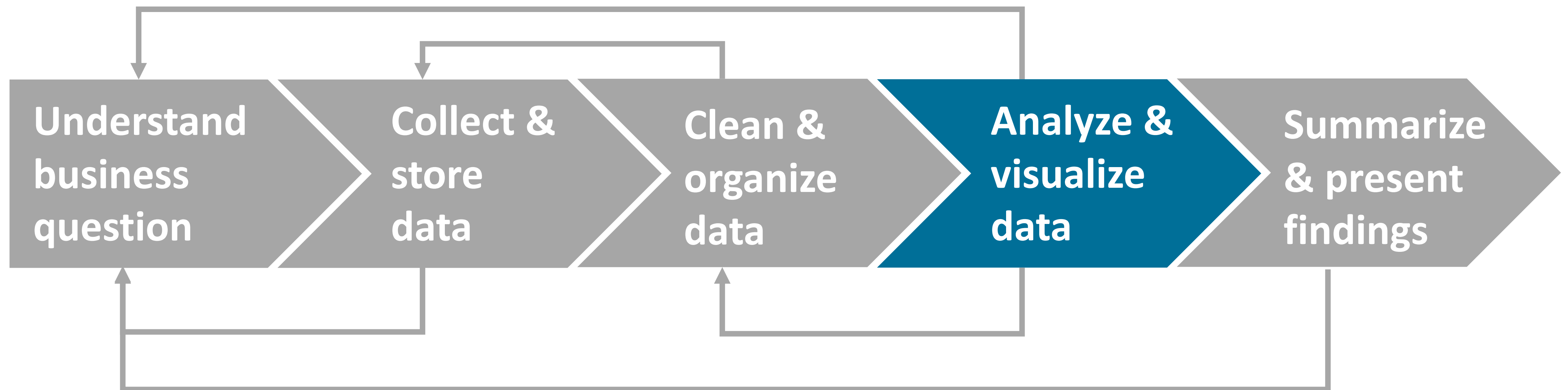
Recap

- **Jupyter notebooks** integrate code and output into a single document with markdown text to help make your work more transparent, understandable, repeatable, and shareable
- **Pandas** is a powerful library that allows us to create series and DataFrames to perform data analysis
- Pandas allows us to easily **read files...**
 - `pd.read_excel('path/filename.xlsx')`
 - `pd.read_csv('path/filename.csv')`
 - `pd.read_pickle('path/filename.pickle')`
 - you can find more via Google (such as `read_sql`, `read_json`)
- ...and **write files** for data analysis purposes
 - `dataframe_name.to_excel('path/filename.xlsx')`
 - `dataframe_name.to_csv('path/filename.csv')`
 - `dataframe_name.to_pickle('path/filename.pickle')`
 - you can find more via Google

Next week

Next week SPRING BREAK! no lecture

In 2 weeks (3/21): **exploring and plotting data** on Zoom



Questions?